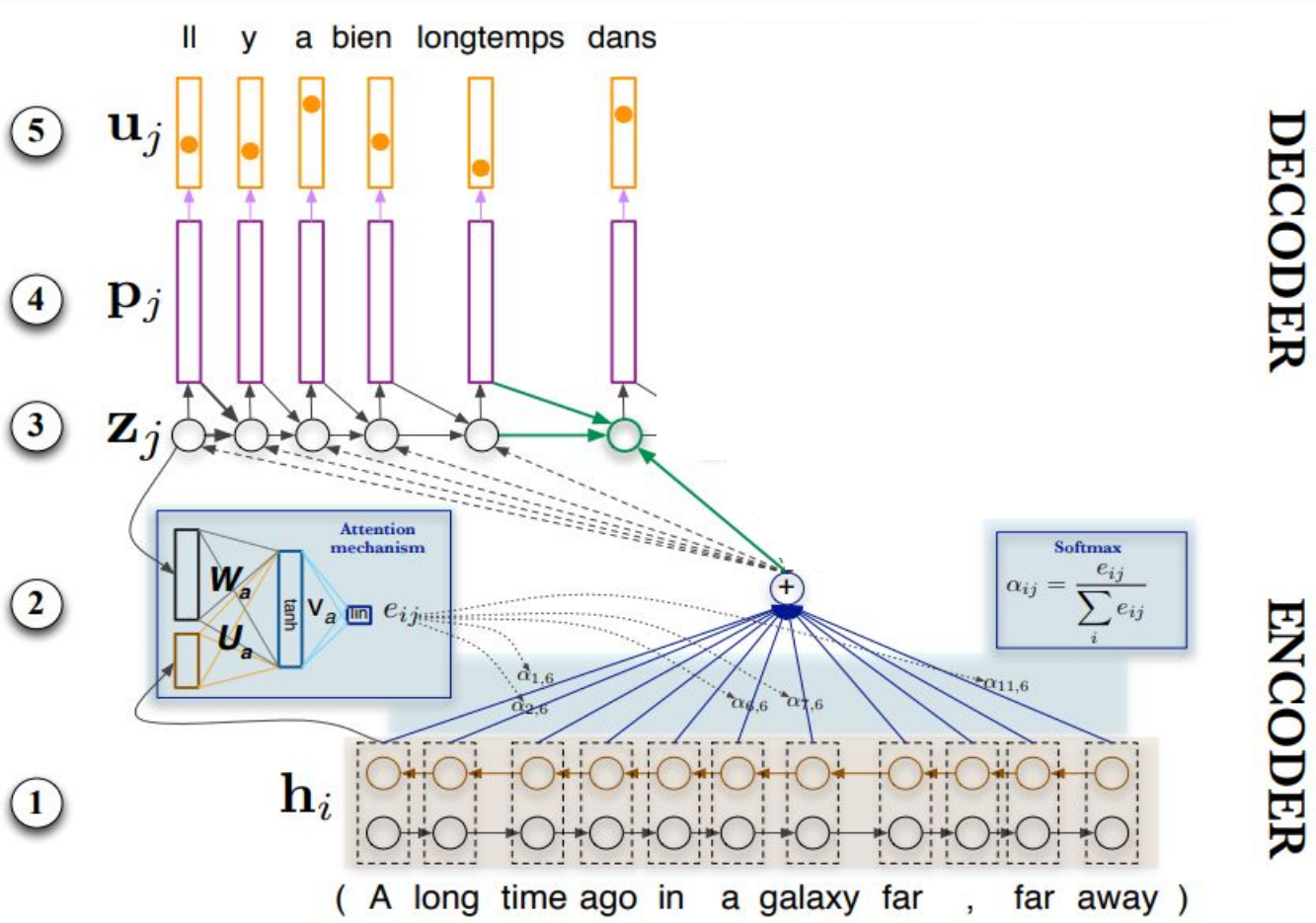
Attention is all you need (Transformer)

Md Shad Akhtar
shad.akhtar@iiitd.ac.in

INDRAPRASTHA INSTITUTE of INFORMATION TECHNOLOGY DELHI

Attention Mechanism [Bahadanu et al., 2015]

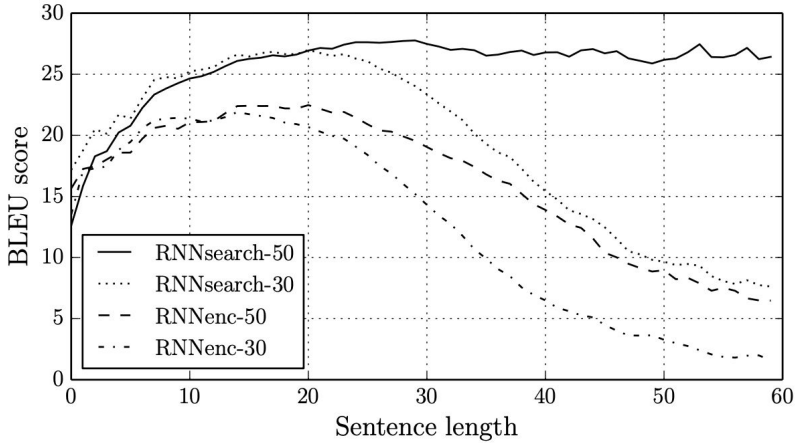
1. Bi-RNN Encoder
2. Attention
3. RNN Decoder
4. Output Embeddings
5. Output probabilities

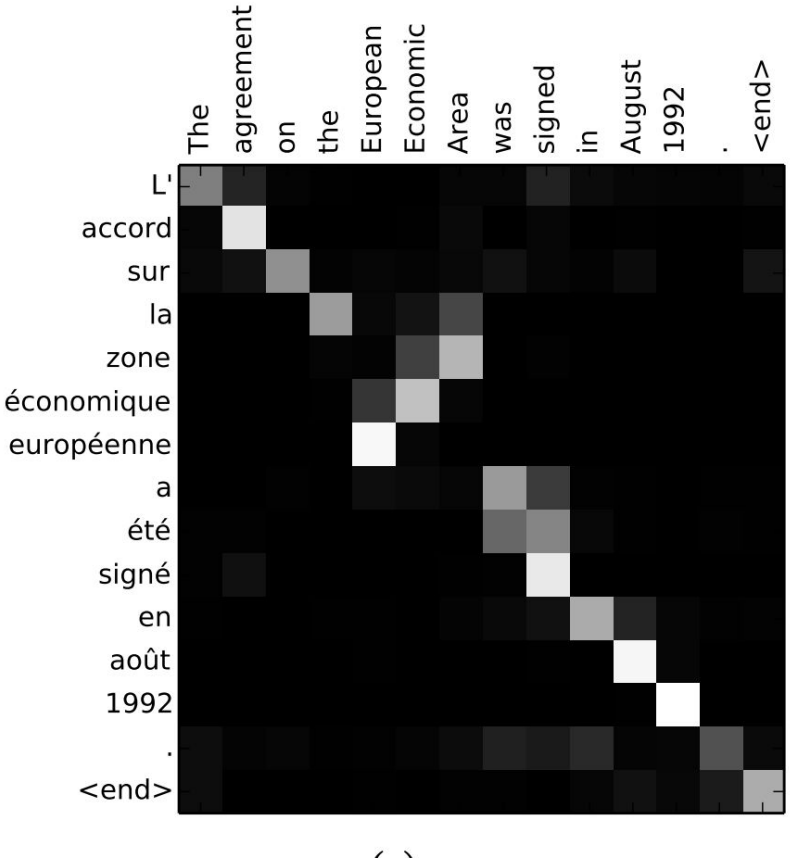


[Garc´ıa-Mart´ınez et al., 2016]
[Bahadanu et al., 2015]

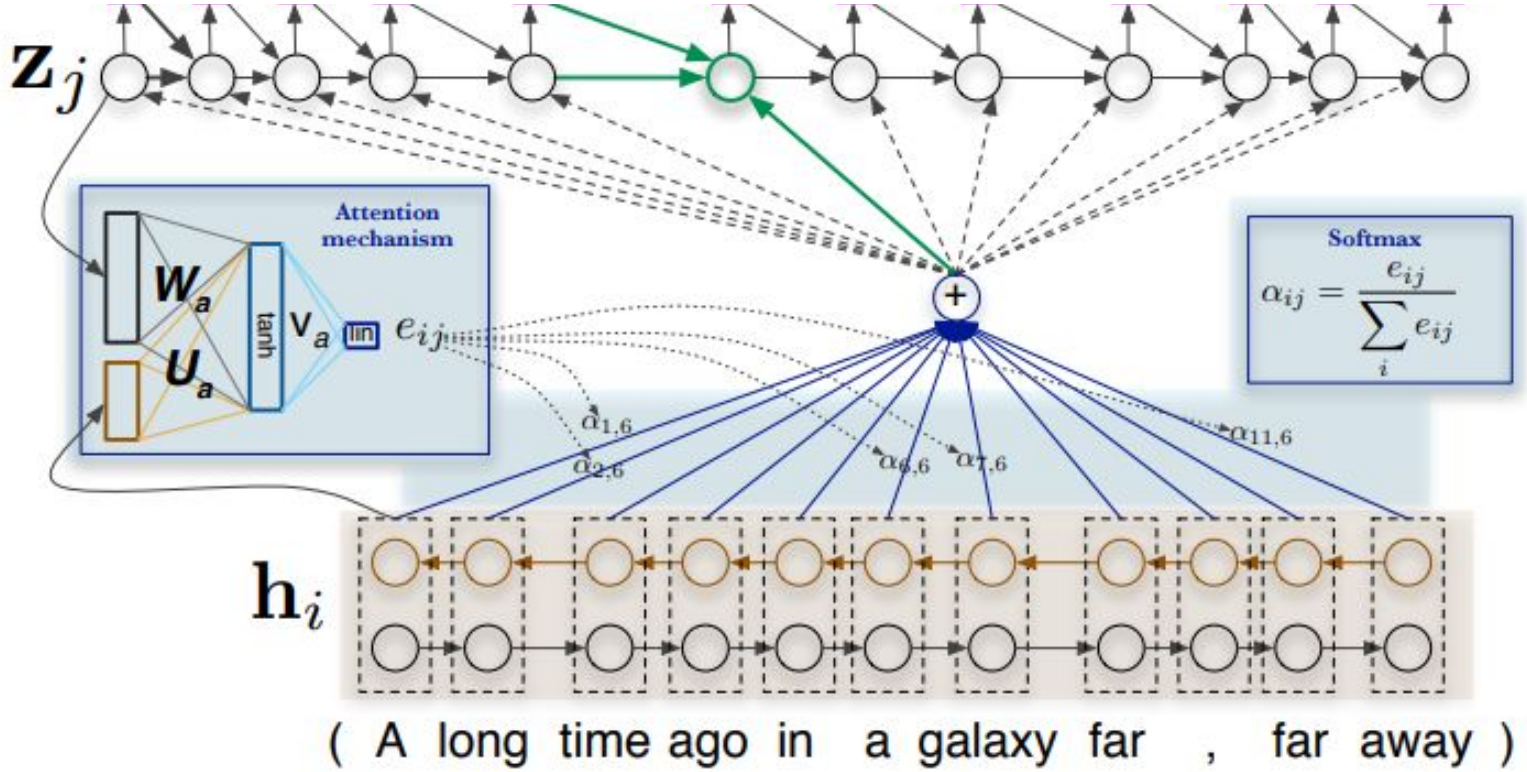
Results

- English-French WMT'14 dataset
- RNNenc-N (without attention)
- RNNsearch-N (with attention)





Attention Mechanism



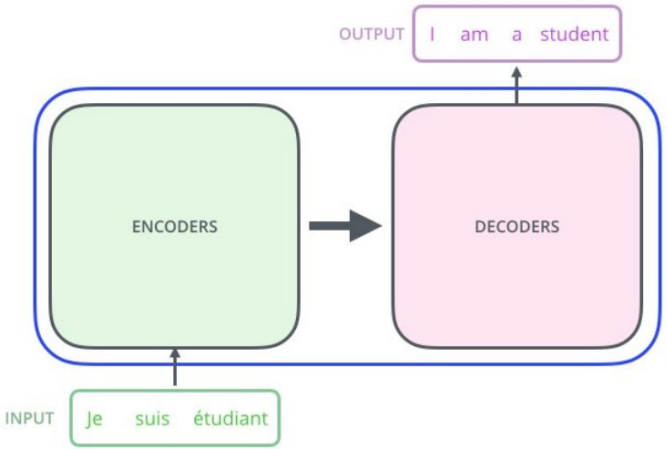
[Garc´ıa-Mart´ınez et al., 2016]
[Bahadanu et al., 2015]

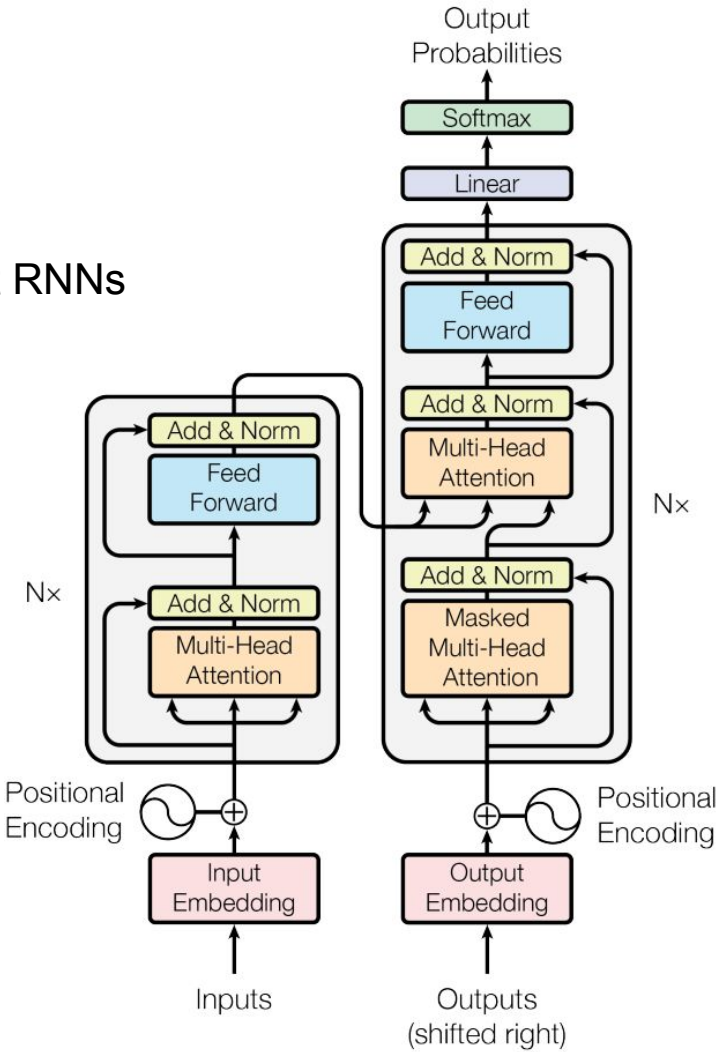
Encode-Attend-Decode using RNNs: Key points

- Recurrent computation is *sequential*!
- This *prevents parallelization* within training examples

Transformer

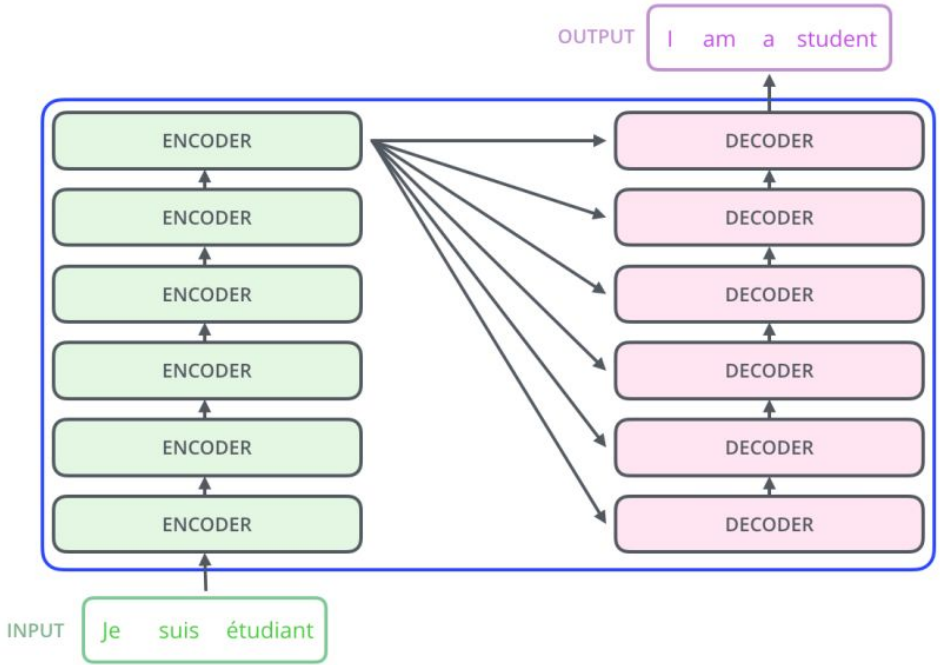
- Its an *encoder-attend-decoder* approach, but without RNNs





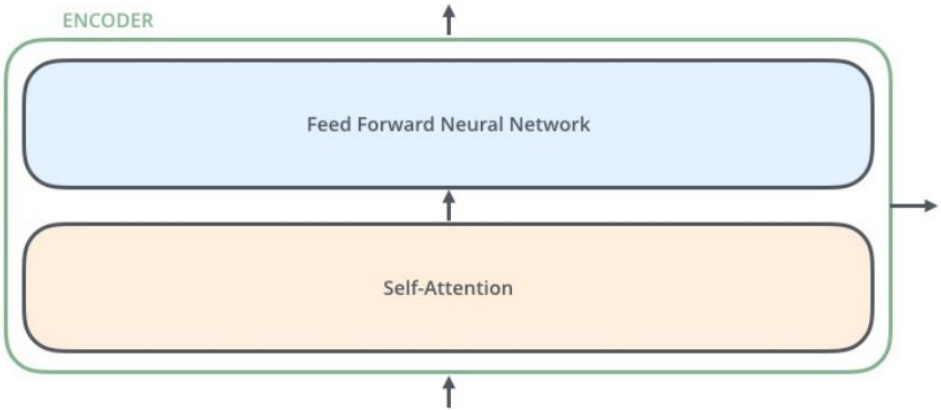
Transformer

- Six encoder and six decoder layers

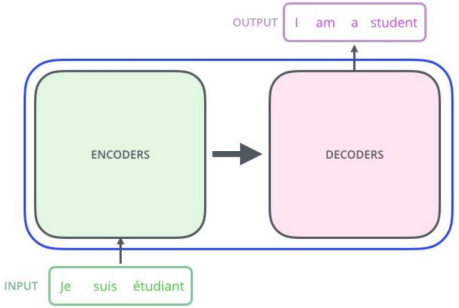


Transformer

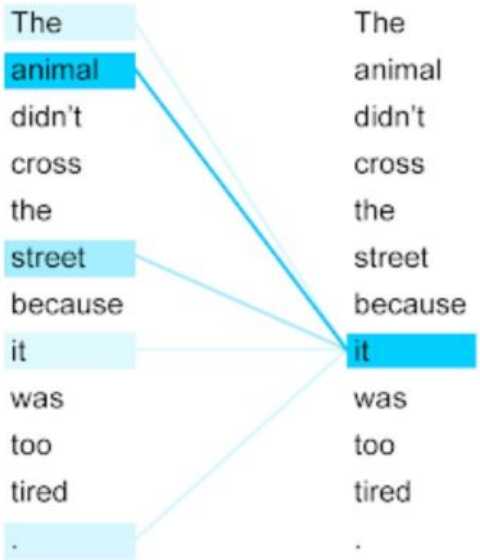
- Encoder has two sublayers
  - Self Attention
  - Position-wise FFN

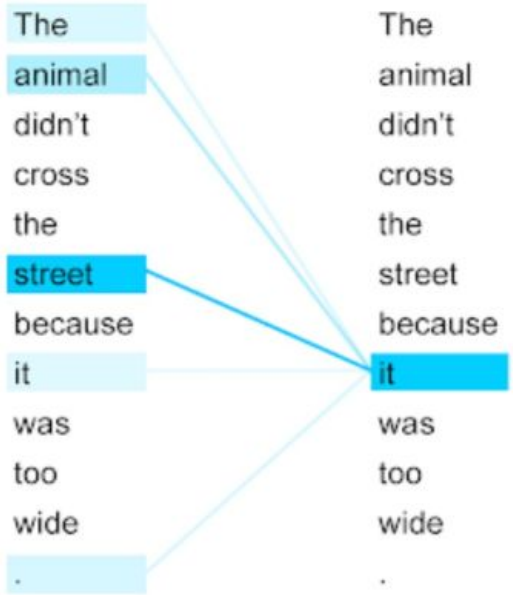


Self-Attention



- Compute representation of a word in a sequence considering the relevance of other words in the same sequence.

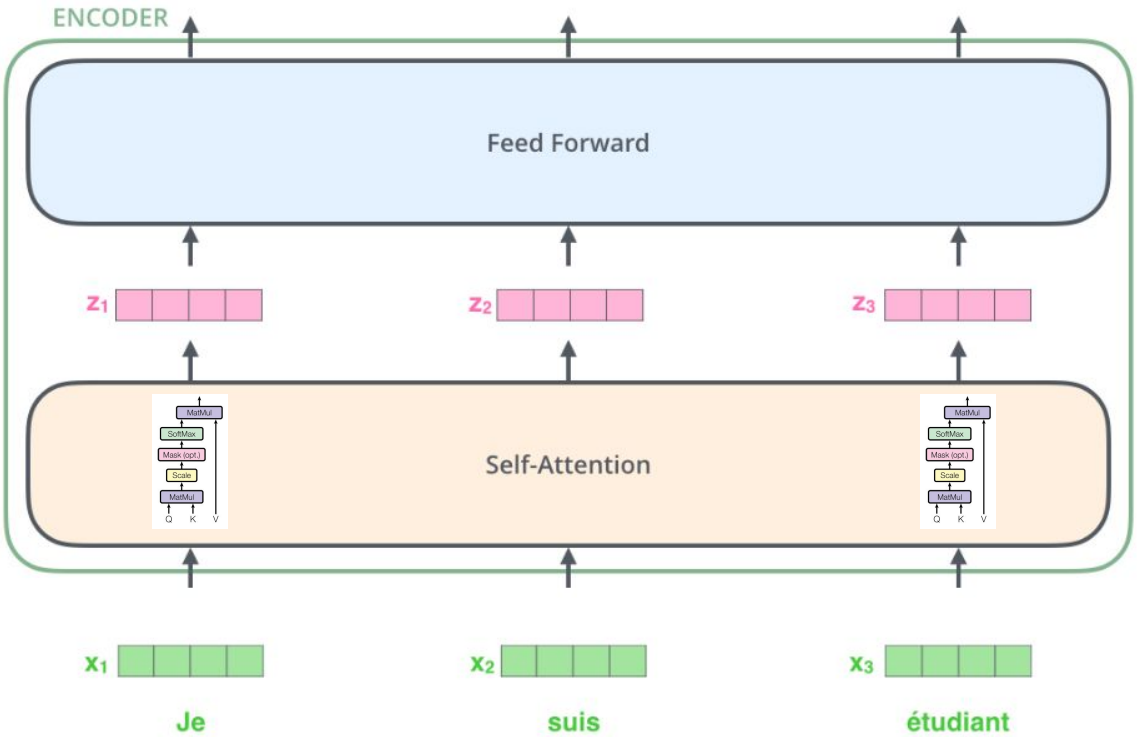


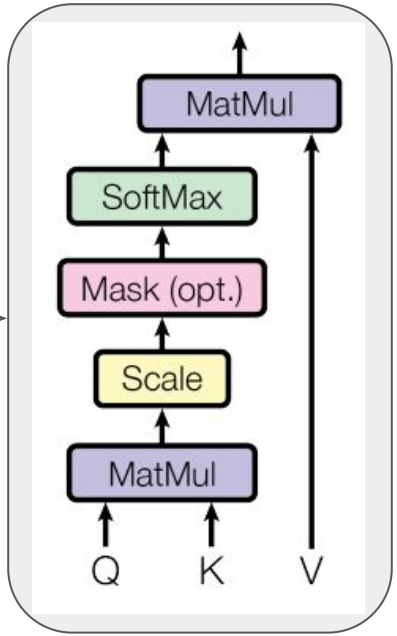


Self-Attention

Input: Fixed-length vector x_i

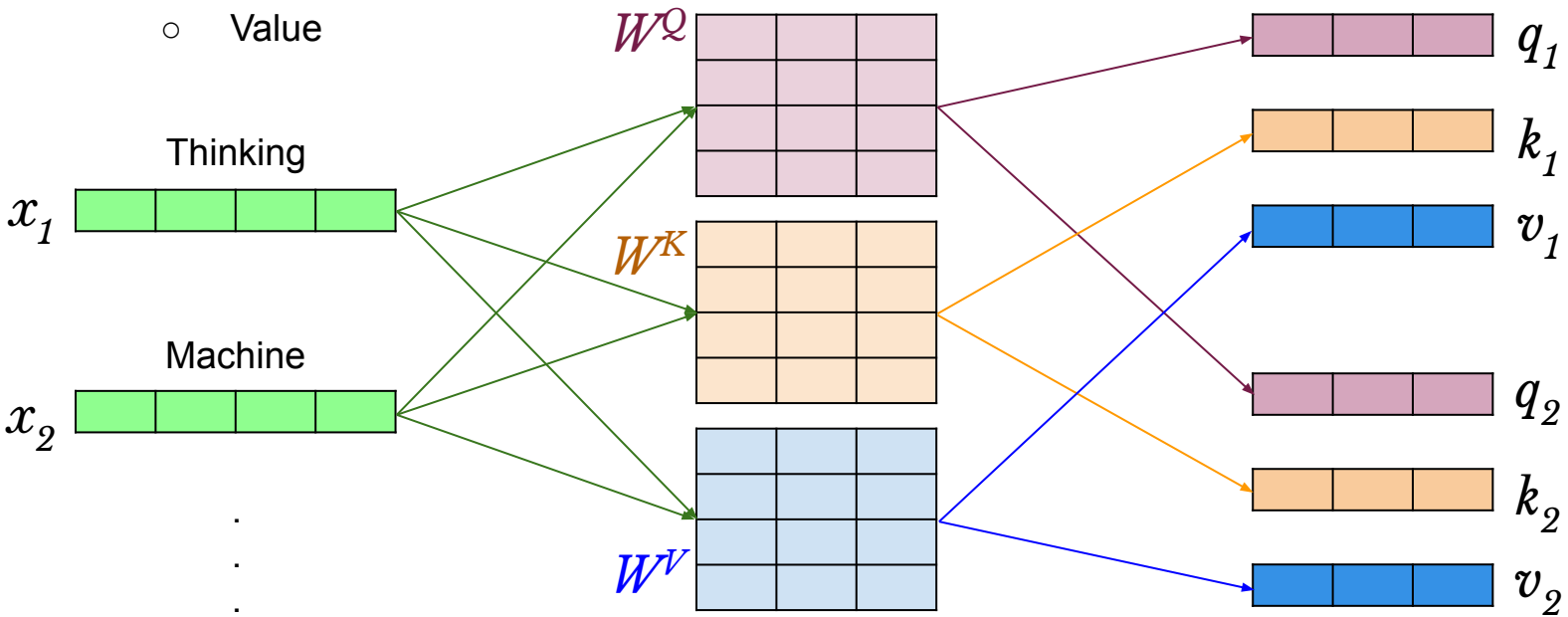
Output: A vector of same length z_i





Self Attention

- Three representation for each input token x_i
  - Query
  - Key
  - Value



Thinking

Machine

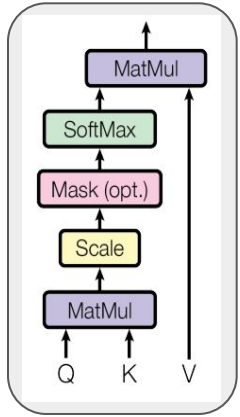
Input x_1 x_2

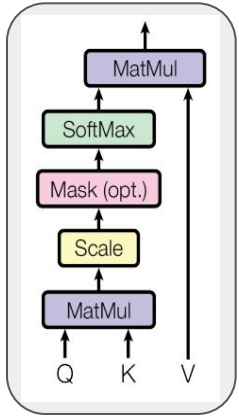
Projection W^Q W^K W^V

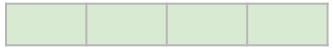
query q_1 q_2

key k_1 k_2

value v_1 v_2

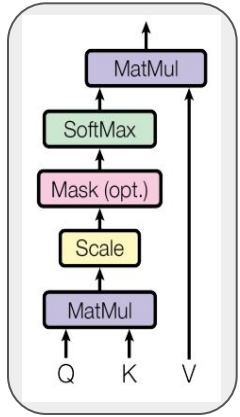




	Thinking	Machine
Input	x_1 	 x_2
Projection	W^Q W^K W^V	
query	q_1 	 q_2
key	k_1 	 k_2
value	v_1 	 v_2
Score for q_1	$q_1 . k_1 = s_{11}$	$q_1 . k_2 = s_{12}$
Scale (divide by $\sqrt{d_k}$)	$s_{11} / \sqrt{d_k}$	$s_{12} / \sqrt{d_k}$
Softmax	*0.88*	*0.12*
Softmax x v_i	v_1 	 v_2
Sum	z_1 	

Thinking

Machine



Input x_1 x_2

Projection W^Q W^K W^V

query q_1 q_2

key k_1 k_2

value v_1 v_2

Score for q_2 $q_2 \cdot k_1 = s_{21}$ $q_2 \cdot k_2 = s_{22}$

Scale (divide by $\sqrt{d_k}$) $s_{21} / \sqrt{d_k}$ $s_{22} / \sqrt{d_k}$

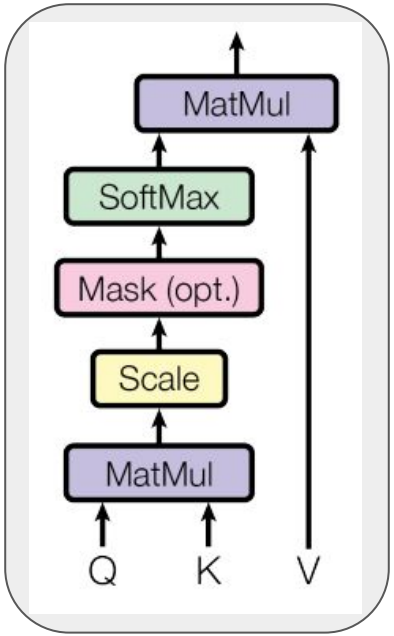
Softmax 0.34 0.66

Softmax x v_i v_1 v_2

Sum z_1 z_2

Attention computation

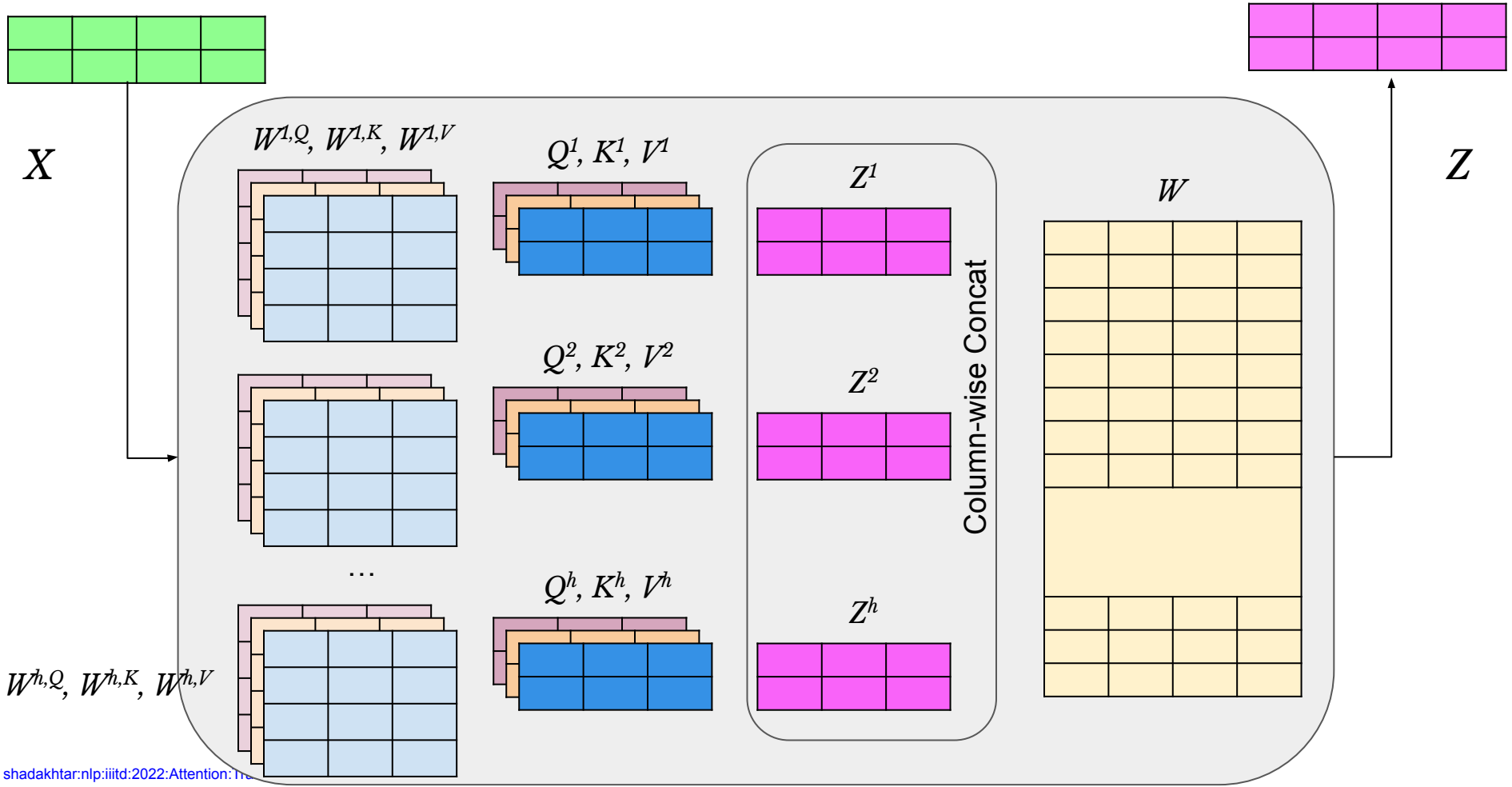
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



Multi-head Attention

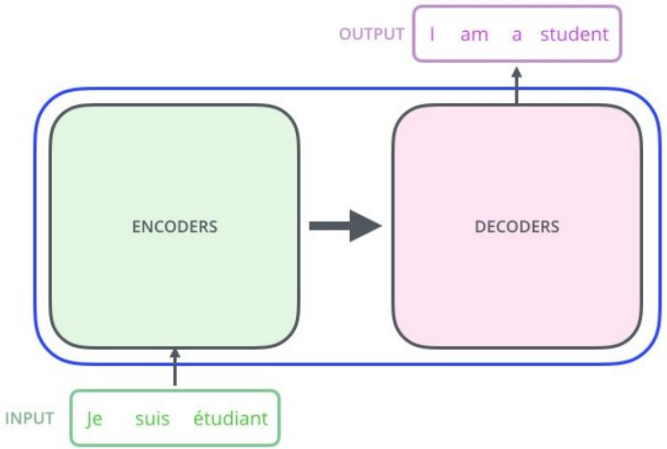
- Dimensions of input and (key, value, query) are not same
  - $d_x > d_{(q, k, \text{or } v)}$
- This doesn't have to be the case, but is an architectural choice
  - In paper,
    - $d_{model} = 512$ and $d_q = d_k = d_v = d_{model} / h = 64$ (i.e., $h = 8$)
- This results in a loss of information, but is made up for with something called multi-head attention
  - Instead of projecting the inputs only once, do it h times

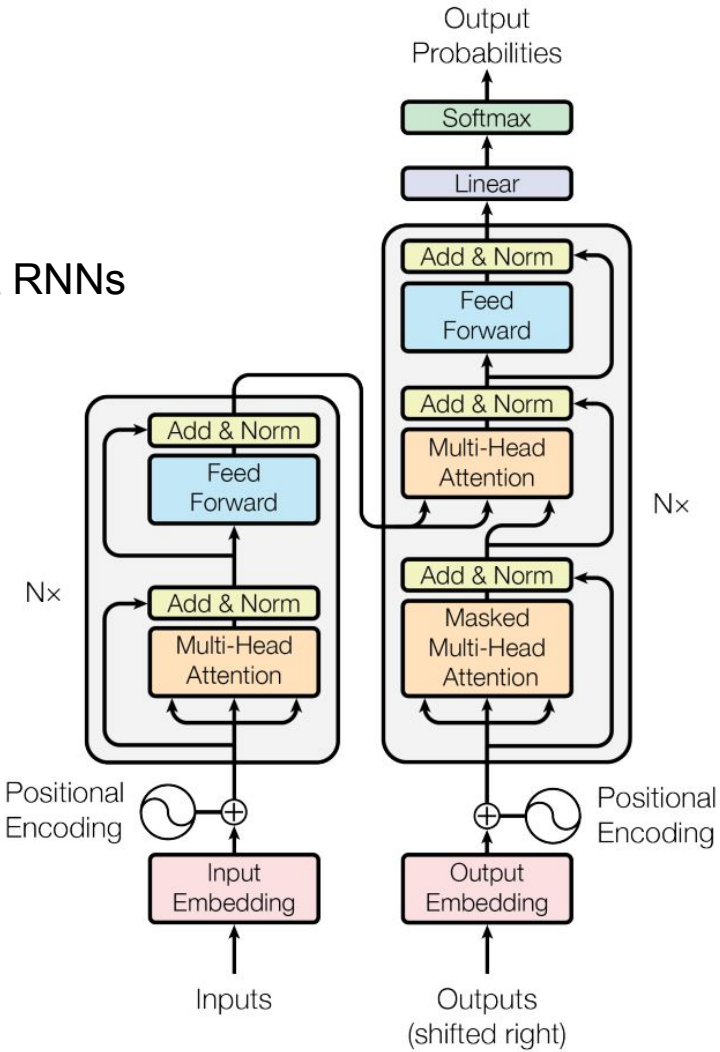
Multi-head Attention



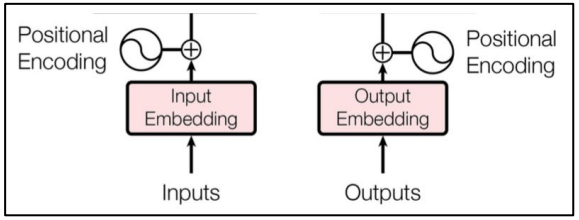
Transformer

- Its an *encoder-attend-decoder* approach, but without RNNs

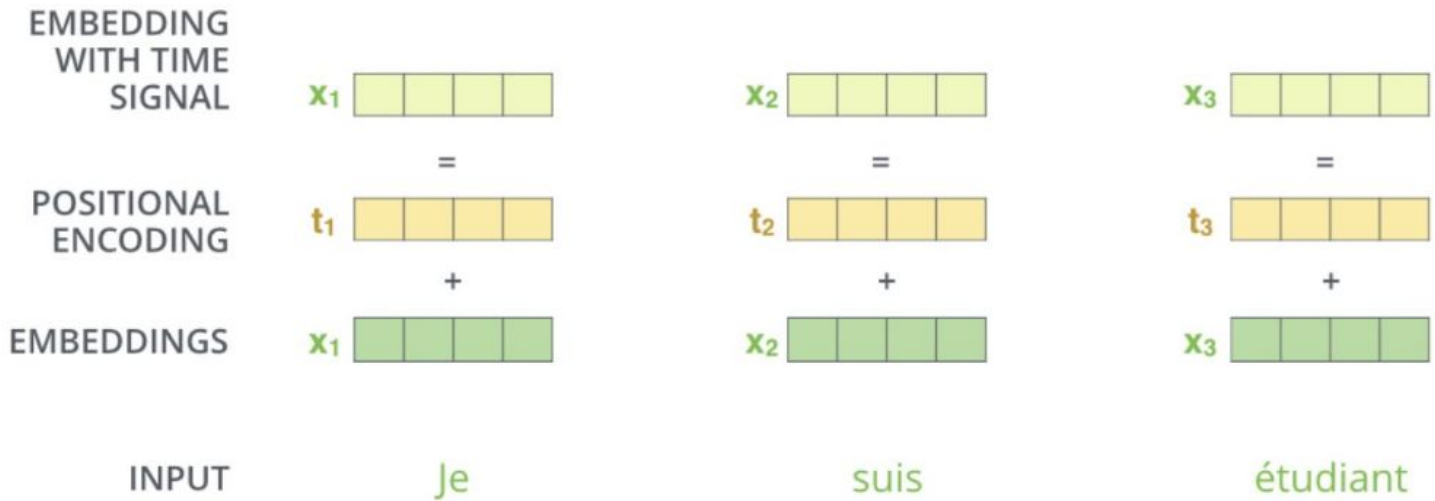




Positional Embedding



- No order of words is known to encoder
- A vector is added to each input embedding, which encodes the relative position of each token

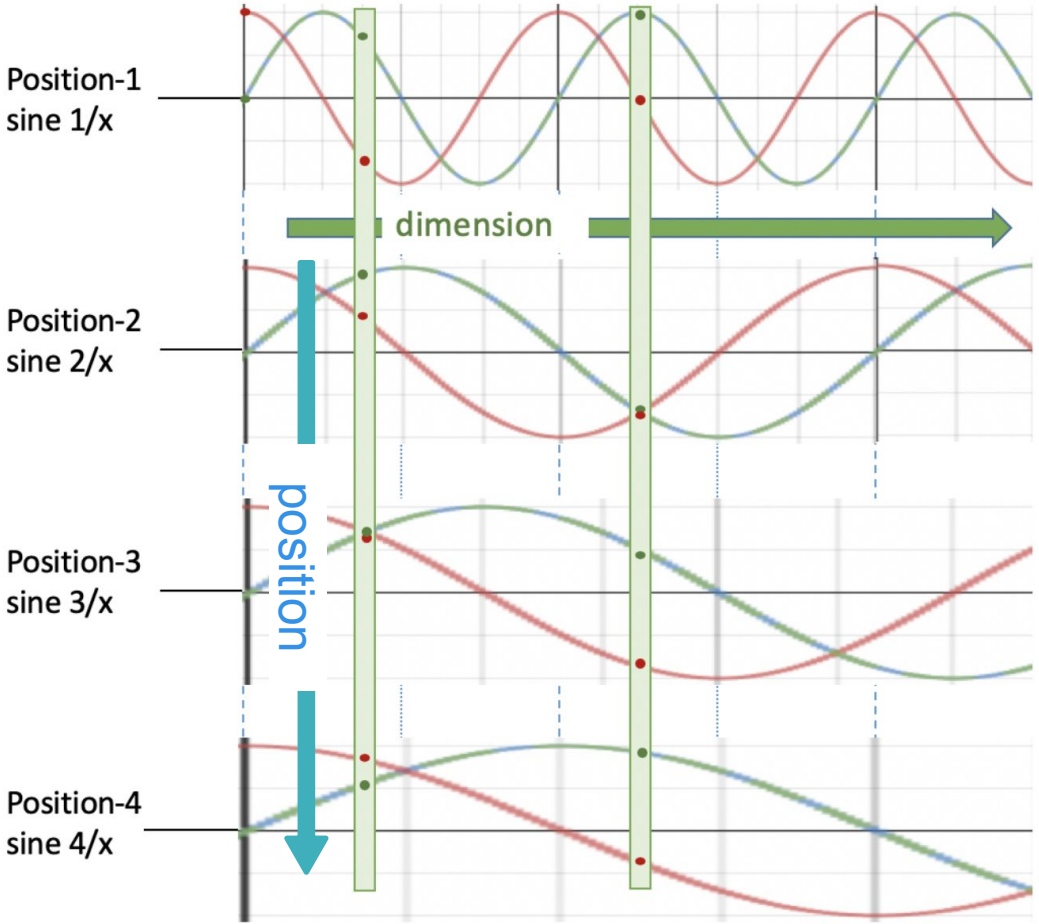


Positional Embedding

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

pos = Token's position
i = dimension

Model will extrapolate to sequence lengths longer than the ones seen during training



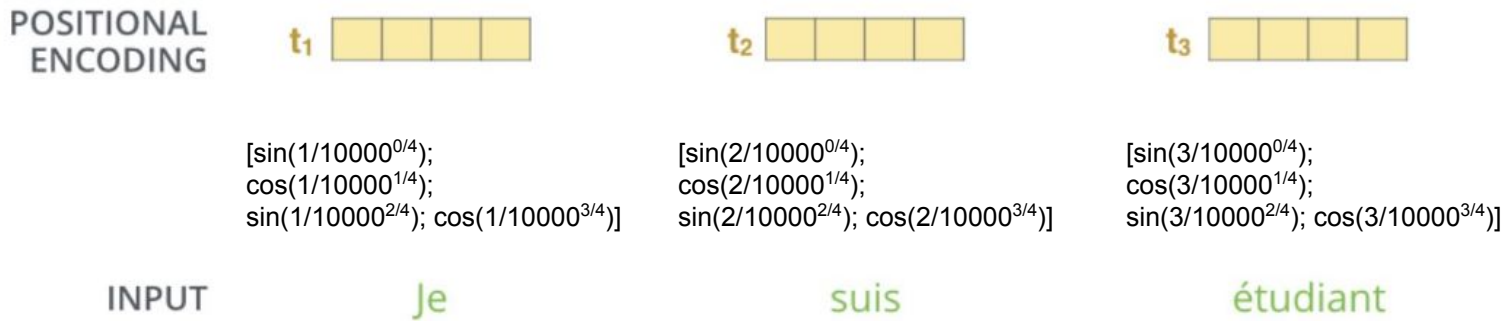
pos = Token's position

i = dimension

Positional Embedding

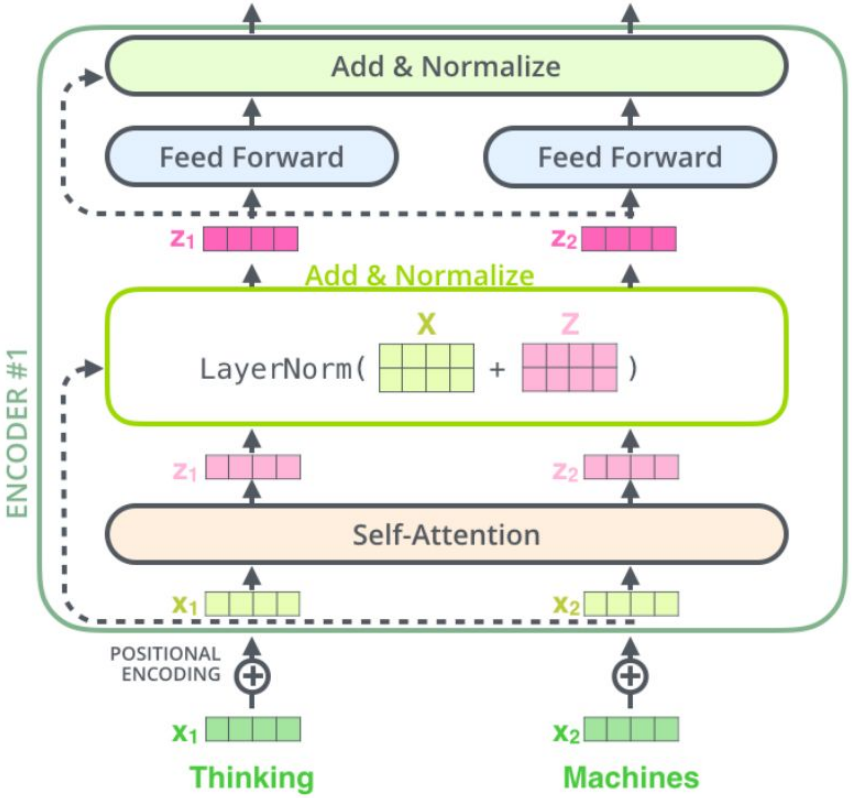
$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$

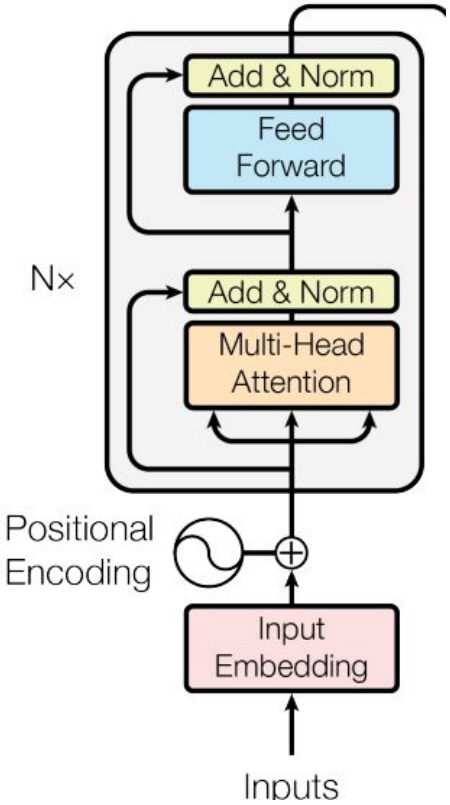
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$



$$\text{FFN}(z) = \textit{ReLU}(zW_1 + b_1)W_2 + b_2$$

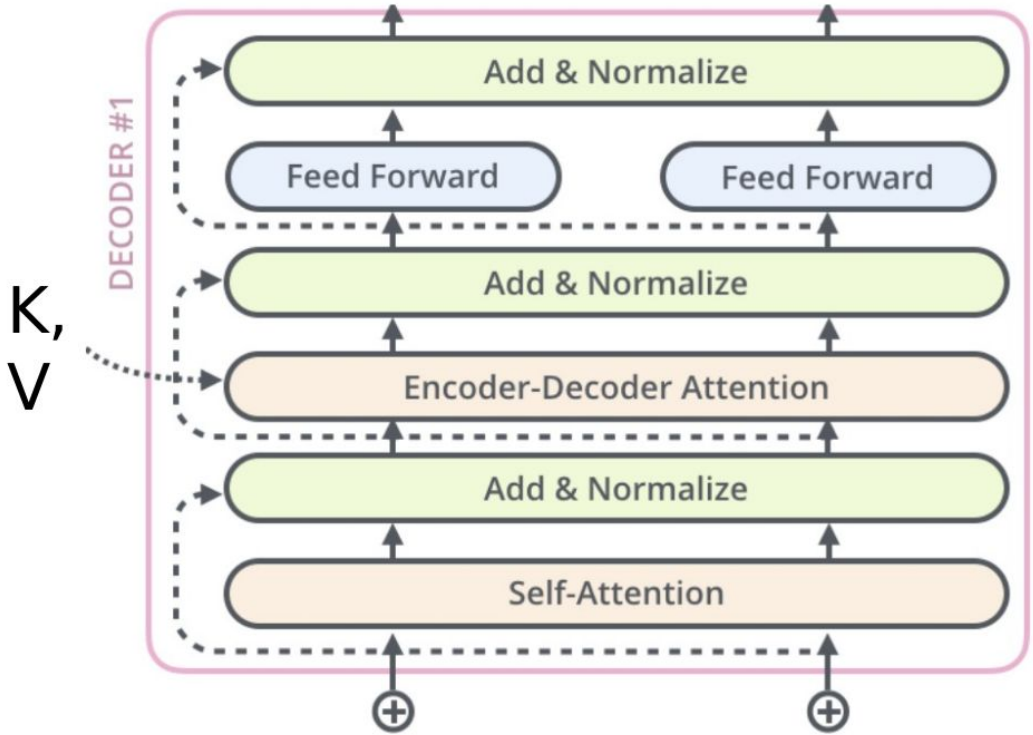
Encoder: Putting everything together



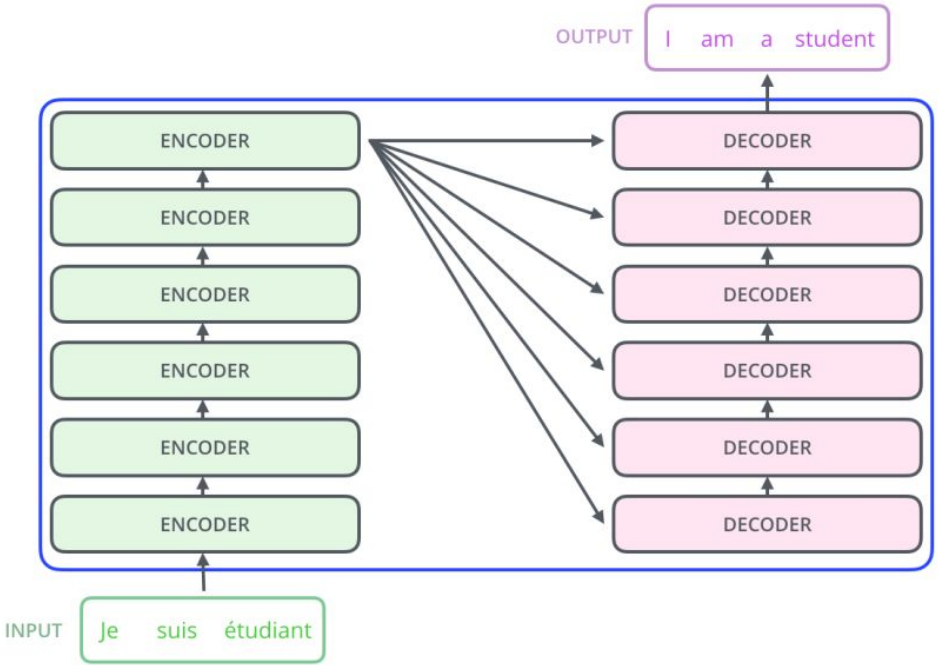


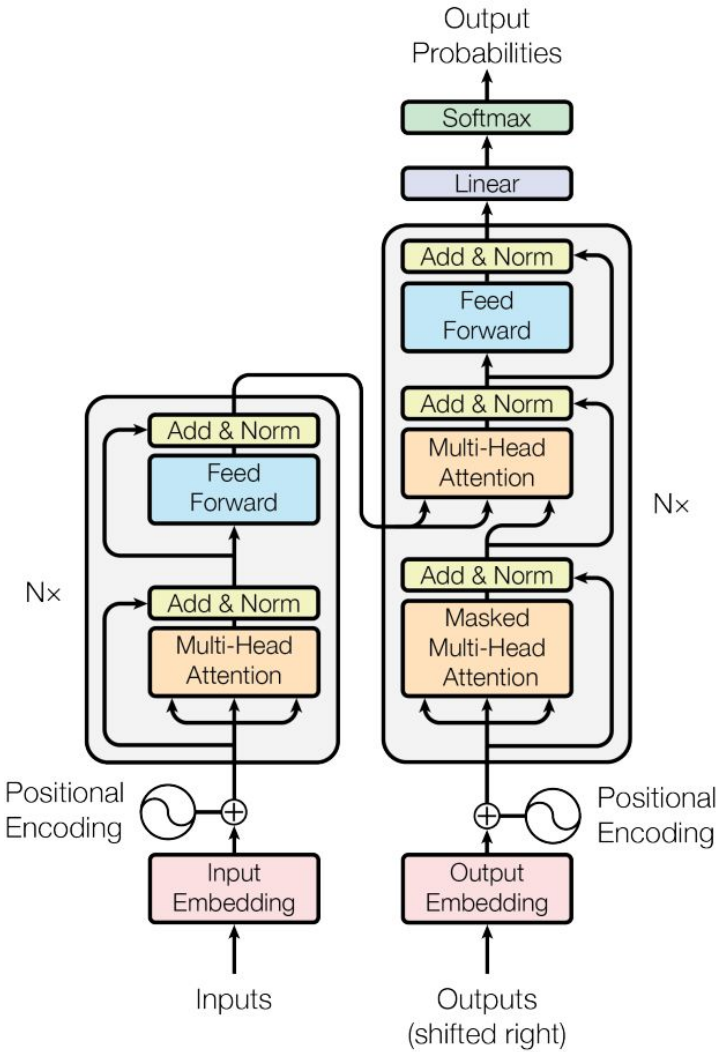
Decoder

- Similar to encoder
- A 3rd sublayer is added
  - Encoder-Decoder attention
- Available outputs with positional encodings are provided as input



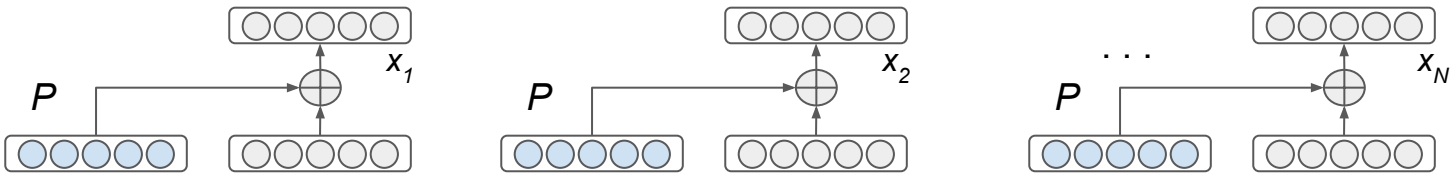
Transformer



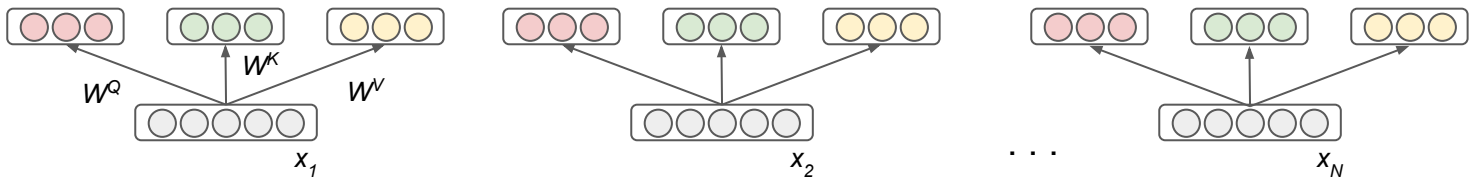


Illustrate Transformer through a network (i.e., neurons, weight connection, etc.) for a single token. Assume the sentence has n tokens. Make other necessary assumptions.

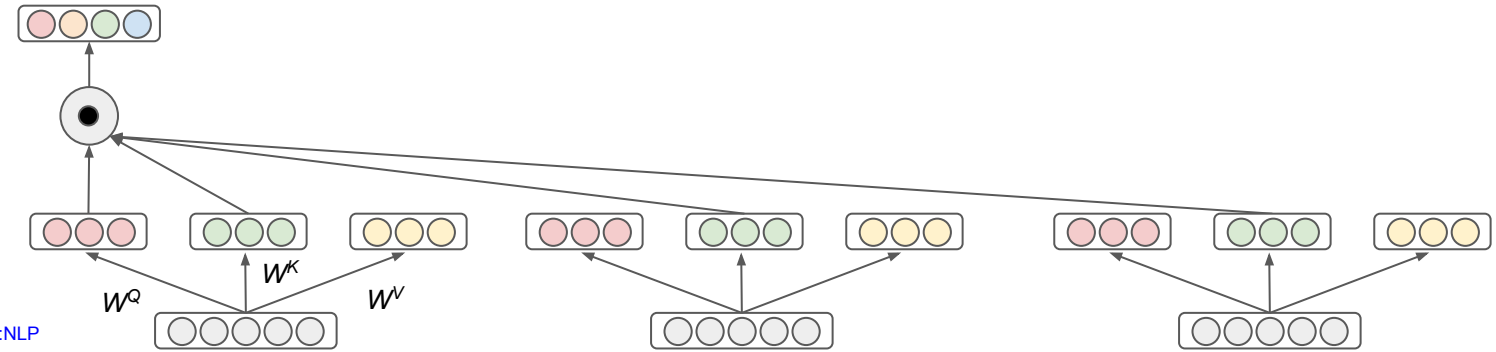
Input + PE



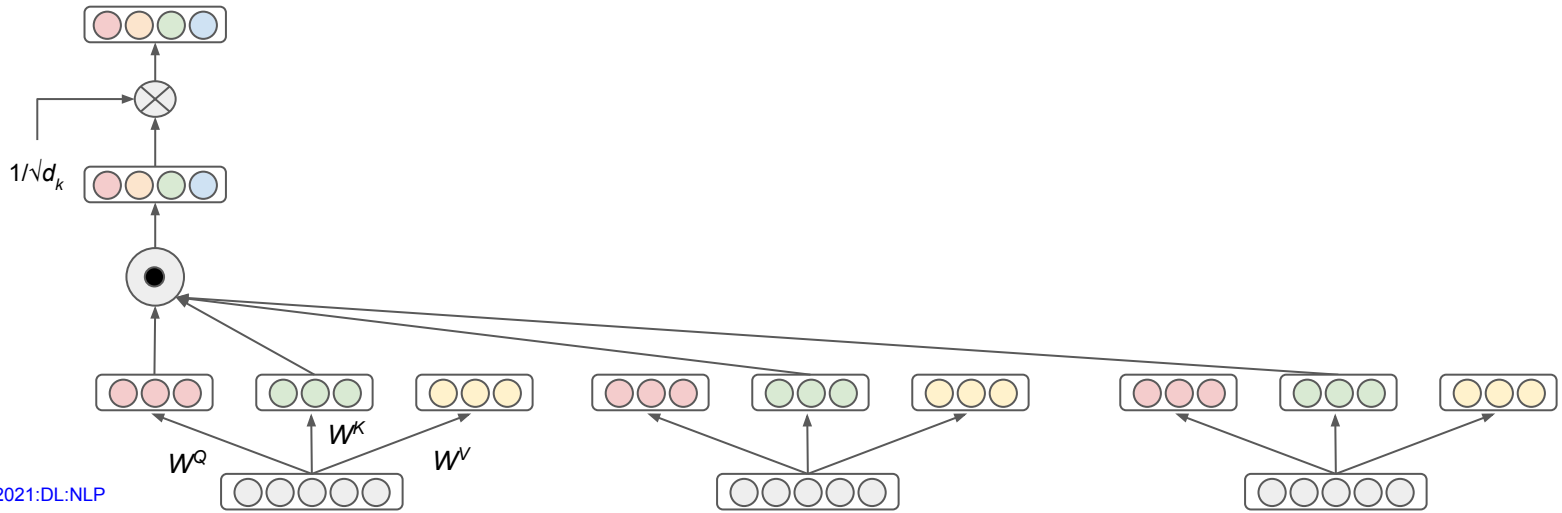
Projection Matrices



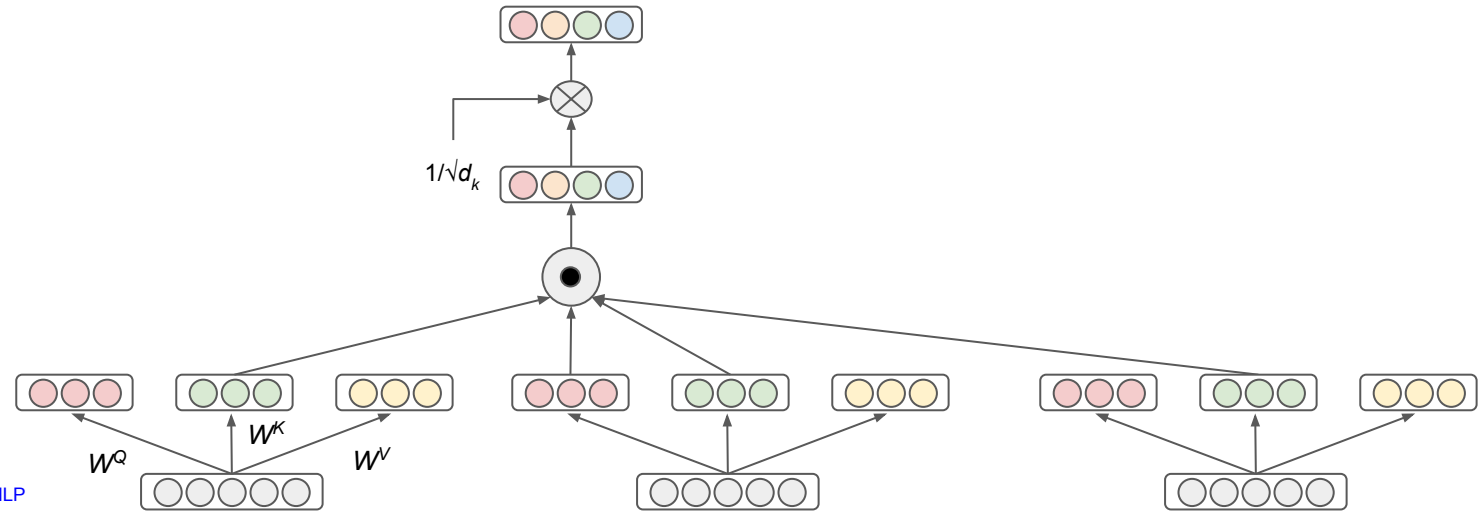
Self Attention: Query*Key



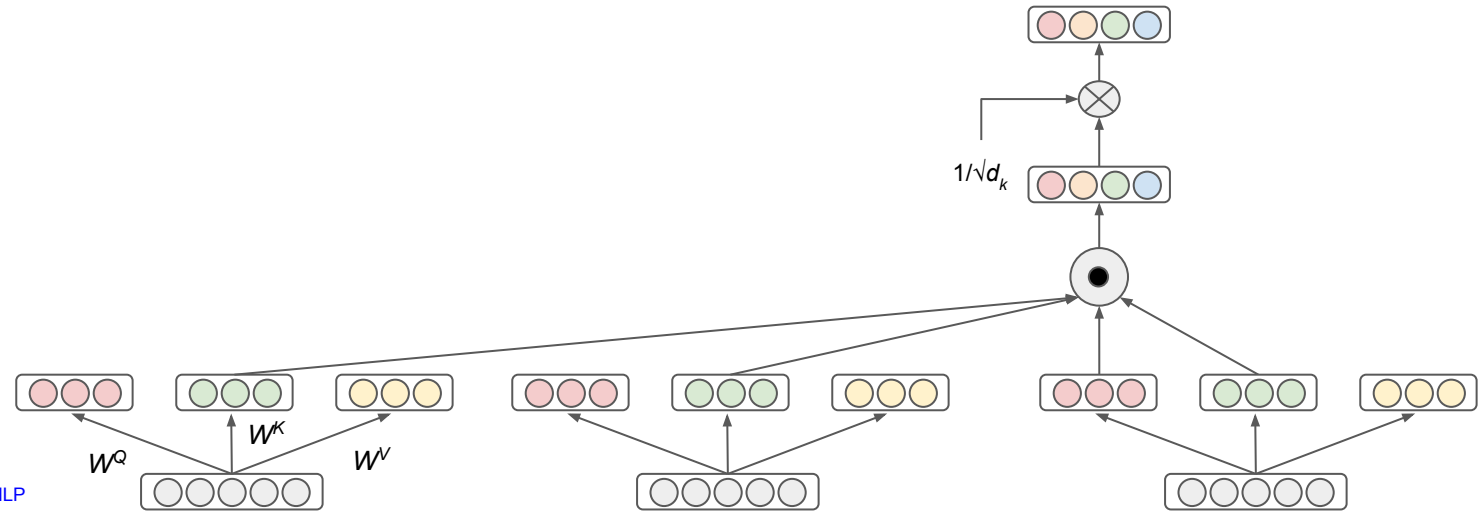
Self Attention: (Query*Key)/$\sqrt{d_k}$



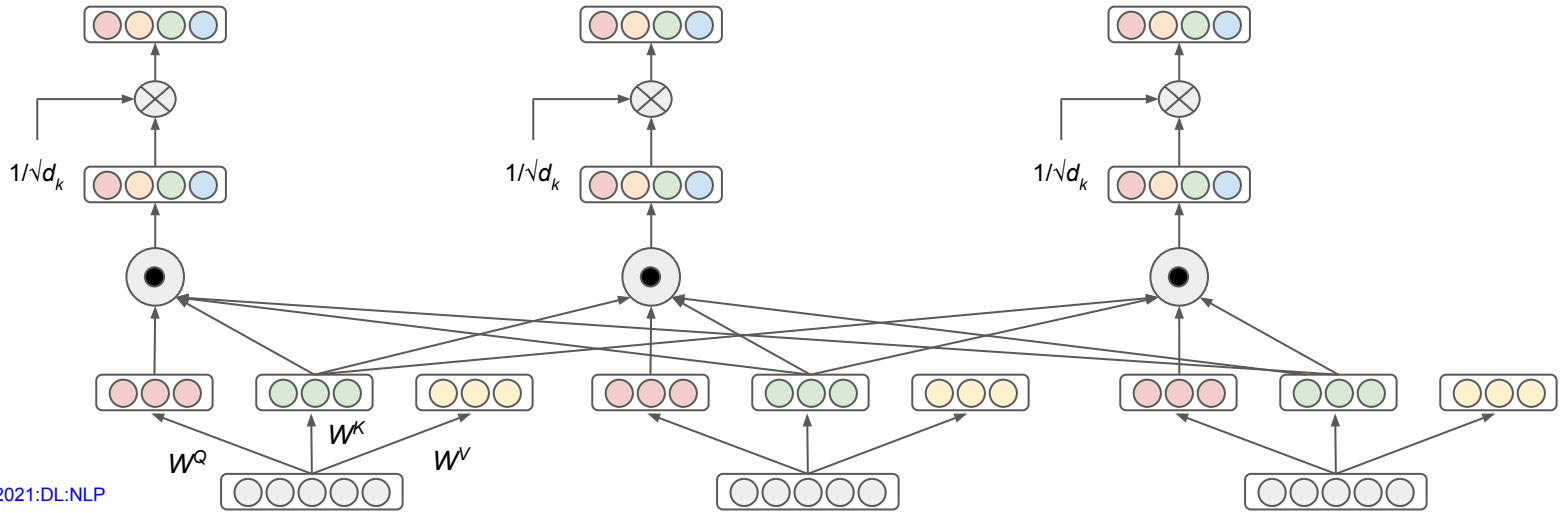
Self Attention: (Query*Key)/$\sqrt{d_k}$



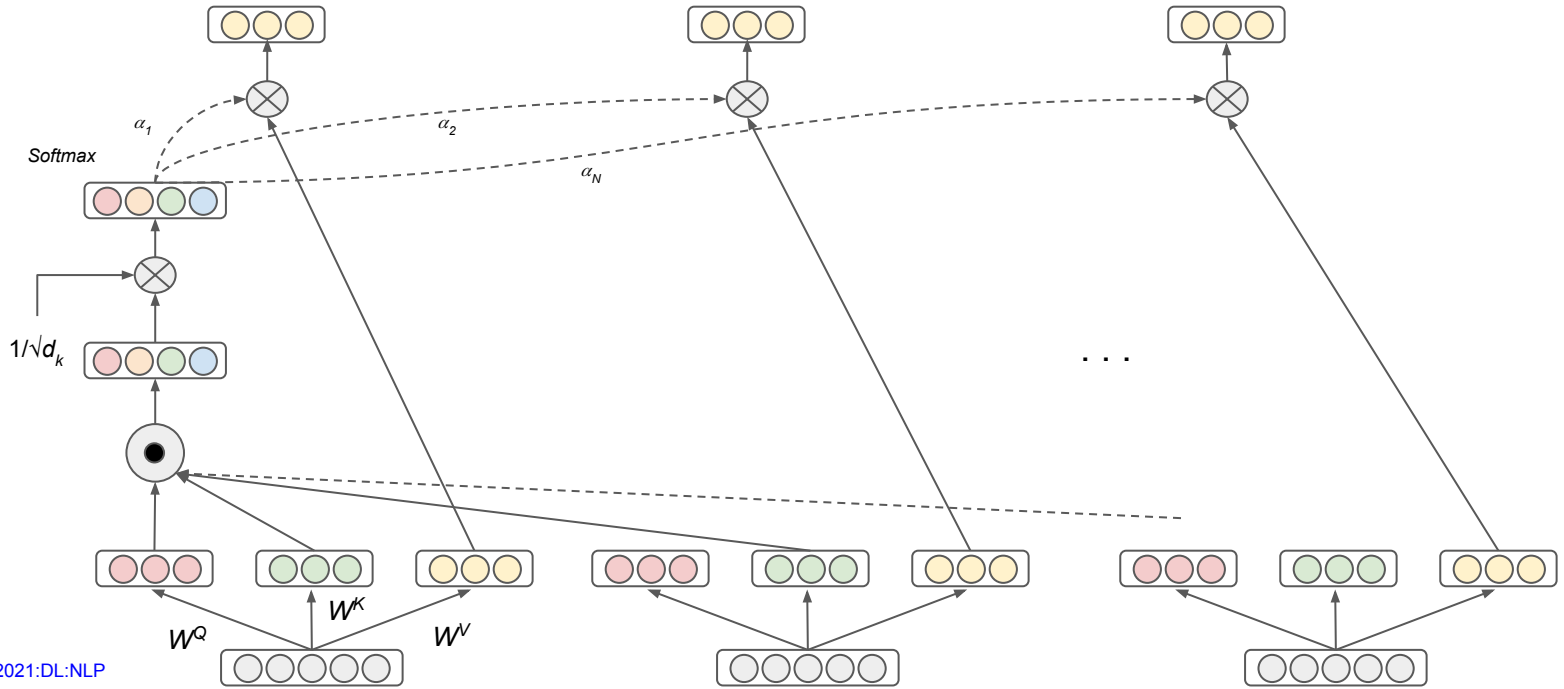
Self Attention: (Query*Key)/$\sqrt{d_k}$



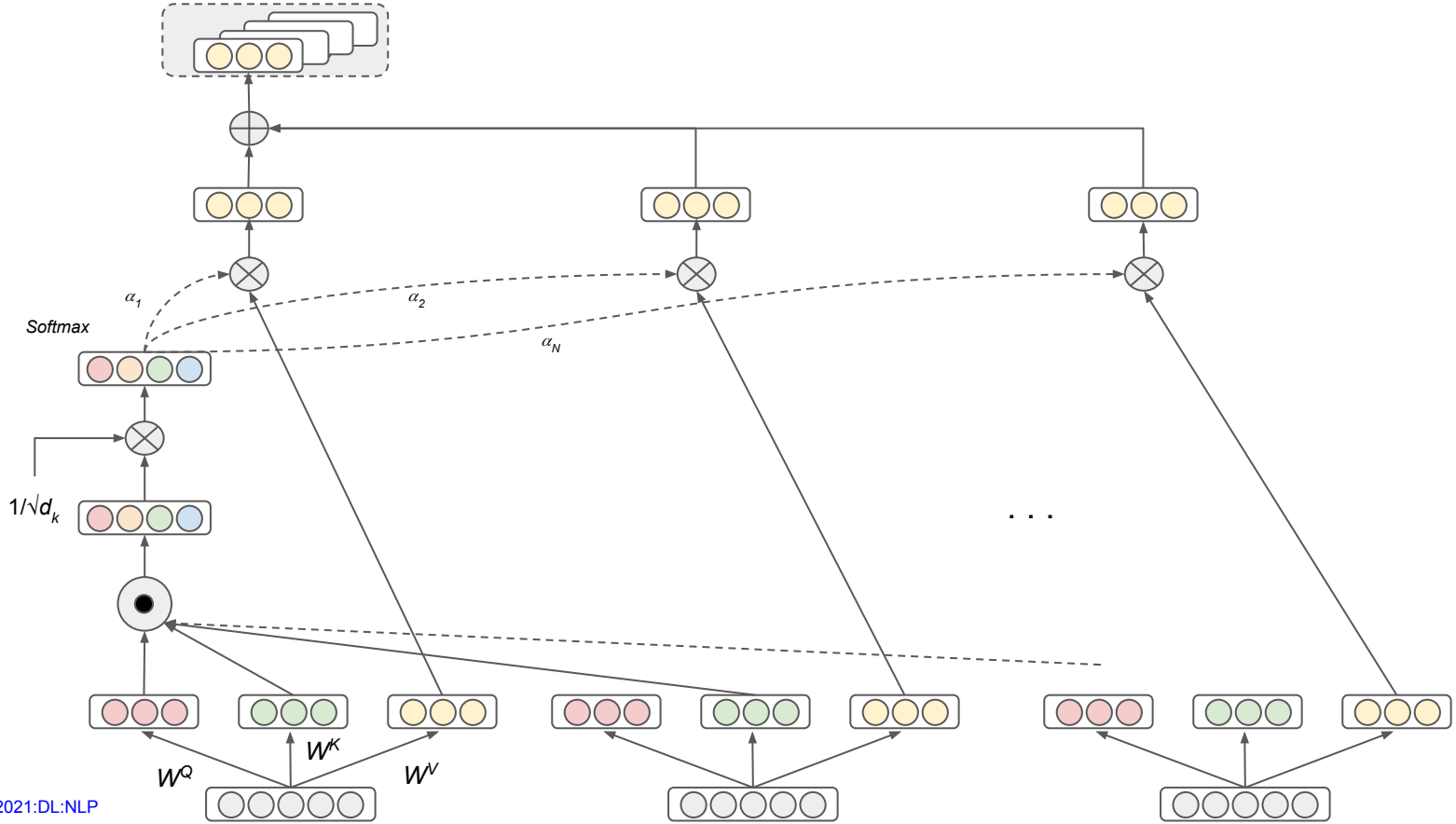
Self Attention: (Query*Key)/$\sqrt{d_k}$



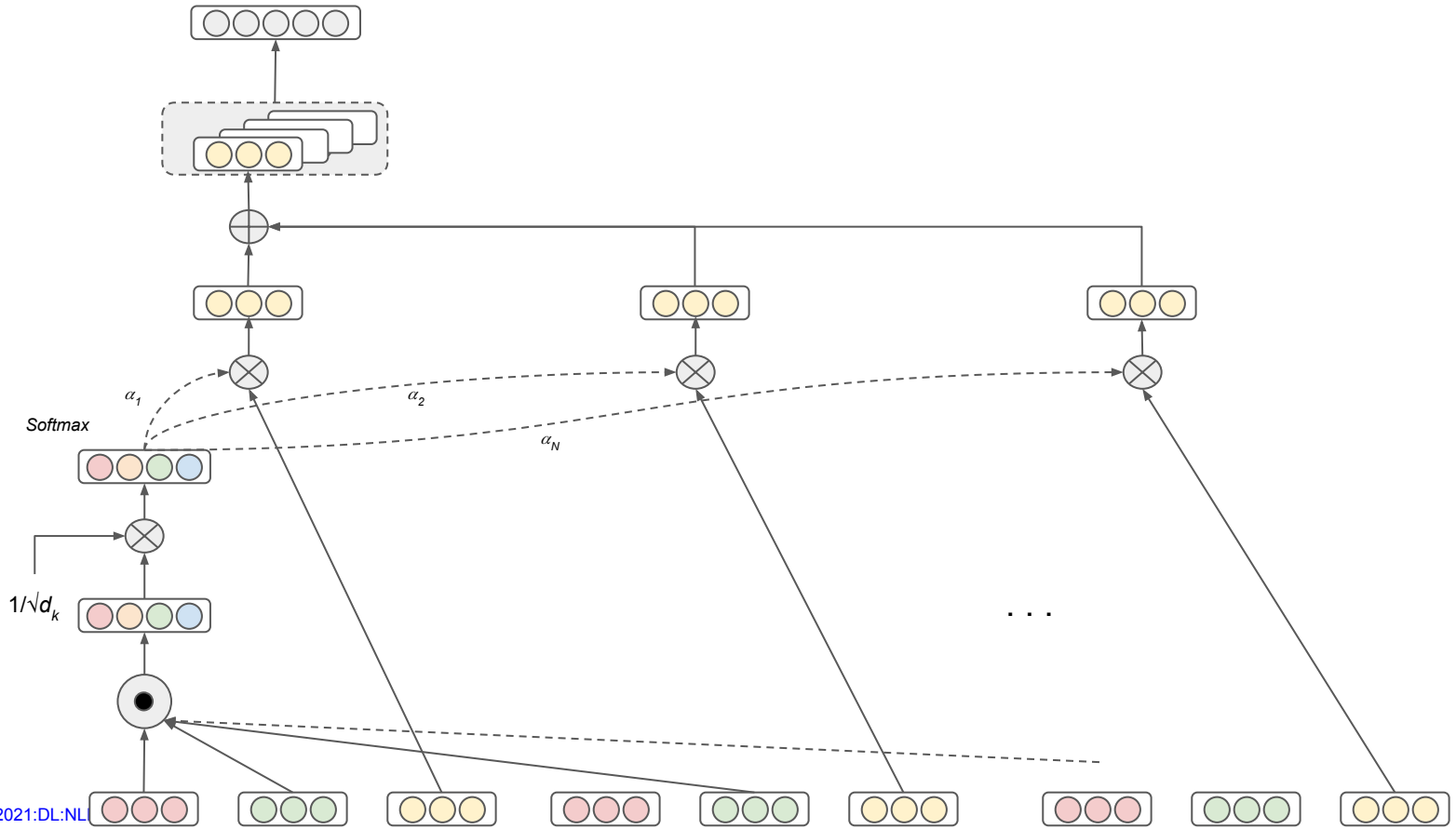
Self Attention: Softmax((Query*Key)/$\sqrt{d_k}$) Value



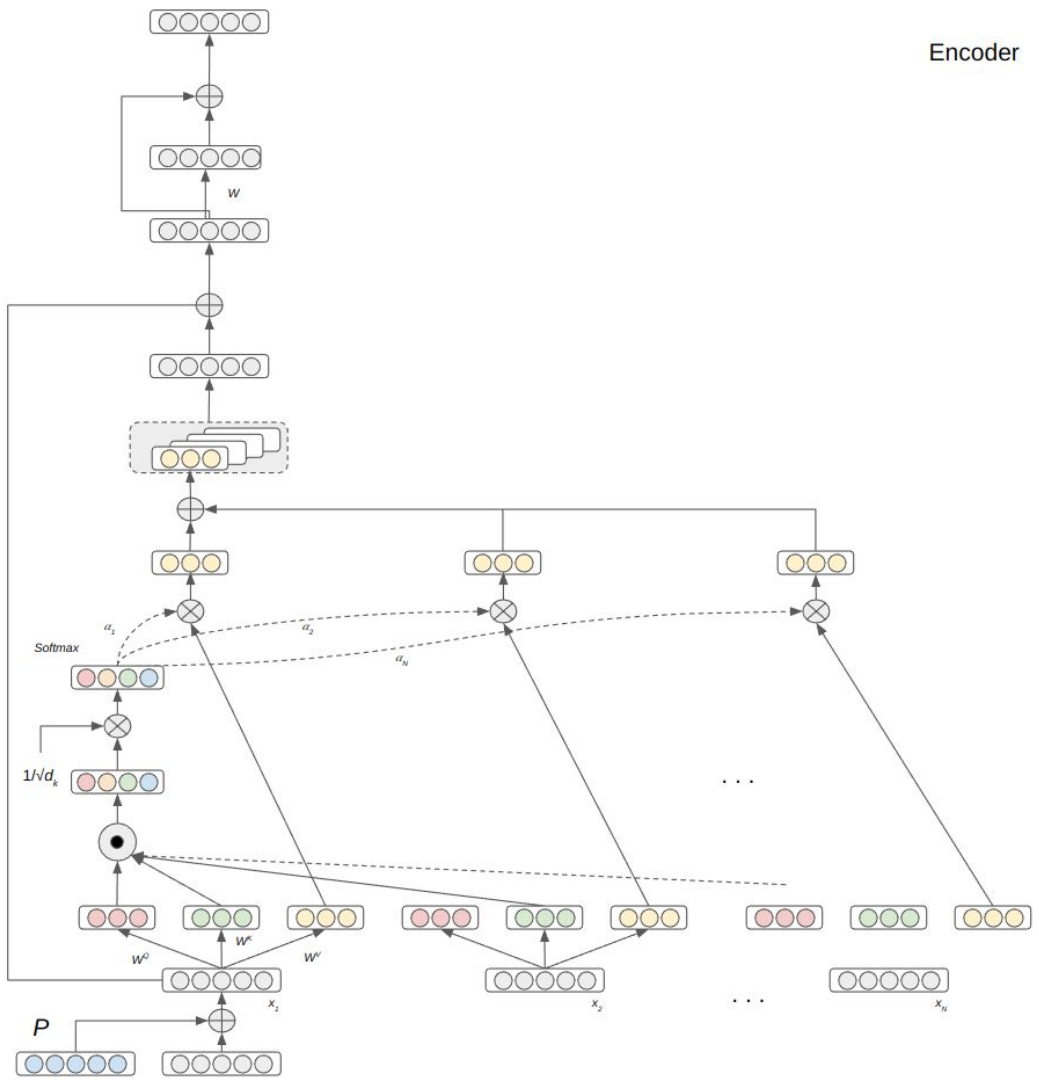
Self Attention: Sum attended values



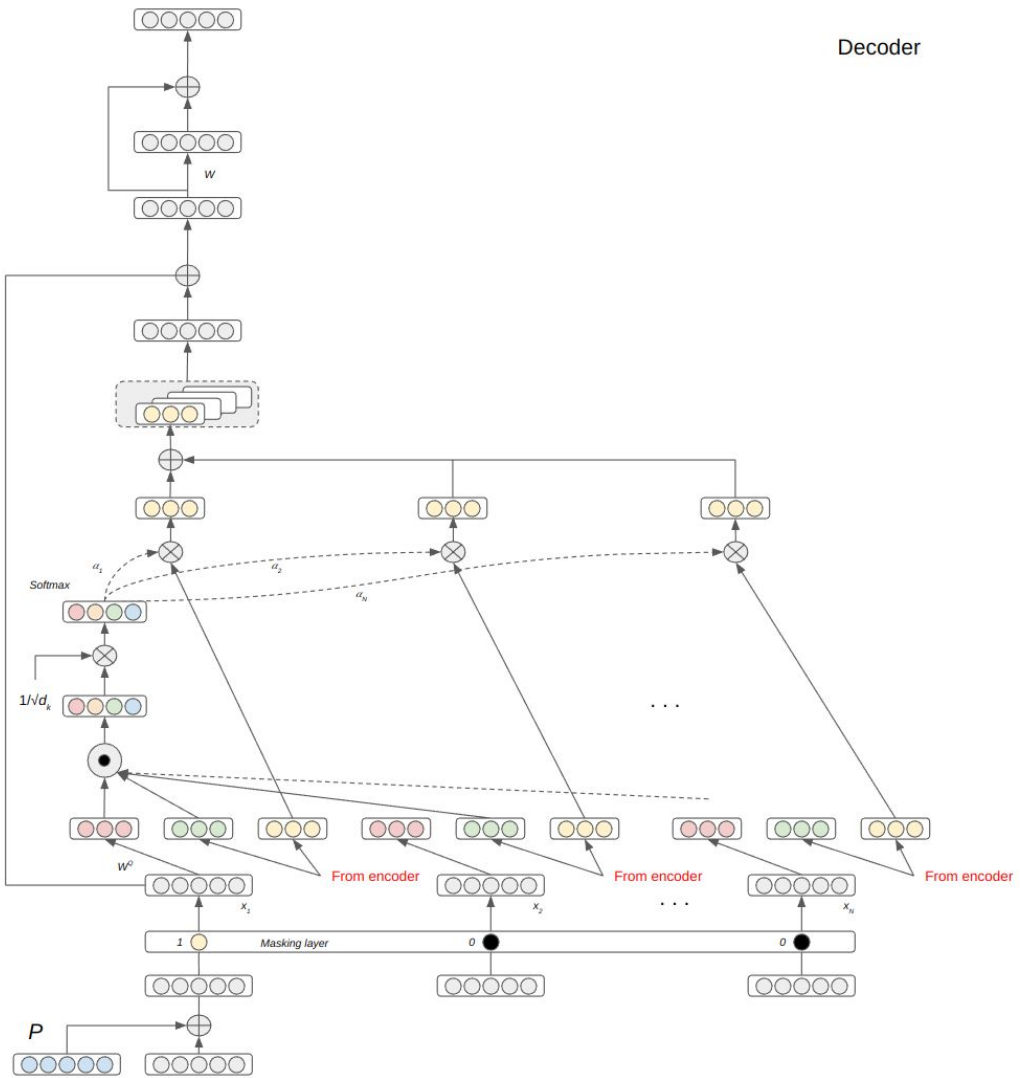
Self Attention: Dimension preservation at output



Encoder



Decoder


Decoder
W
Softmax
α_1
α_2
α_N
$1/\sqrt{d_k}$
W^Q
From encoder
From encoder
From encoder
x_1
x_2
x_N
1
Masking layer
0
0
P